2026 暑假學習報告

文獻探討

Syu

2026/9/16

Index

1. Stream RAG (arXiv 2510.02044)
2. Think While Watching (arXiv 2603.11896)
3. Video Streaming Thinking (arXiv 2603.12262)
4. Adversarial Challenges in NIDS (arXiv 2409.18736)
5. A Defensive Framework Against Adversarial Attacks on ML-Based NIDS (arXiv 2502.15561)

Stream RAG (arXiv 2510.02044)

分別於傳統 SDS (Spoken Dialogue System) 的 Cascaede pipeline: ASR (Automatic Speech Recognition) -> LLM -> TTS，E2E(End-to-End) 雖然較快，但依賴於訓練數據，缺乏事實依據及最新內容，儘管能仿效 text-based 的 conversational assistants 透過外部工具檢索相關資訊以提高準確性，但會造成額外的延遲並打斷對話流暢，本研究旨在使用串流 RAG 提供低延遲的工具呼叫。

Stream RAG (arXiv 2510.02044) — 問題

問題:

1. 基於語音提出的查詢可能不準確
    - sol:
        1. 引入框架工具整合語音 I/O，並實證利用網路搜尋與知識圖形 API 有效助於提升準確性但造成延遲
        2. 對於第一點，以 StreamRAG 與用戶發言平行觸發工具查詢:
            - Fixed-interval Stream RAG: 定期觸發工具查詢，並檢查檢索結果
            - Model-triggered Stream RAG: 根據用戶語句的演變智能判斷最佳查詢時機
2. 可能觸發不必要的工具呼叫
    - sol: AudioCRAG (測試集) 用於評估延遲與準確率

Stream RAG (arXiv 2510.02044) — 做法

做法:

- 整合 LLM 與工具
  1. Query Generation:
     1. 接收使用者語音輸入 Q
     2. 根據 posterior distribution (事後機率) $P(Q^T|Q)$ 產生查詢 (生成關鍵字)
  2. Response Generation:
     1. 根據 Q^T 搜尋得到 R
     2. 根據 posterior distribution $P(A|Q,R)$ 產生語音回應 A

Stream RAG (arXiv 2510.02044) — 做法：Fixed-interval Stream RAG

- Stream RAG: 最小化工具呼叫造成的延遲
  1. Fixed-interval Stream RAG:
     1. $Q = \{Q_b | b = 1, \ldots, B\}$
     2. Block-wise prediction strategy: $\hat{Q}_b^T = \arg\max_{Q_b^T} P(Q_b^T | Q_{1:b})$ (使 P 最大的 Q_b^T)
     3. $b^* = \min\{b \in [1, B] \text{ where reflect } (\hat{Q}_b^T, \hat{Q}_B^T) = \text{True}\}$ (找出第一個區塊 b^*，其發出的早期查詢 $\hat{Q}_{b^*}^T$ 與最終查詢 $\hat{Q}_B^T$ 能得到相同的檢索結果)
     4. 找到 b^* 後終止工具呼叫並使用 b^* 以 $P(A|Q, R_{b^*})$ 產生 R_{b^*} 生成 A
     5. 檢查早期與最終結果是否一致:
        - 兩者前 5 個網頁文件吻合
        - 兩者 KG 一致
     6. 缺點: 依賴外部、運算開銷高

Stream RAG (arXiv 2510.02044) — 做法：Model-triggered Stream RAG

2. Model-triggered Stream RAG: 優化前者的觸發時機 (自動學習)
    1. 學習機制:
        - $\hat{Q}_b^T = \arg\max_{Q_b^T} P(Q_b^T | Q_{1:b}, \hat{Q}_b^{prev})$
        - 標籤生成:
        - $\hat{Q}_b^T = \begin{cases} \text{NO_QUERY}, & \text{if } f(Q_b^T, \hat{Q}_b^{prev}) = \text{True} \\ Q_b^T, & \text{else} \end{cases}$
        - KG 與上一次查詢相同標記為 `NO_QUERY`
        - 與上一次查詢前 5 個網路文件符合標記為 `NO_QUERY`
        - Q_b^T 不是 `NO_QUERY` 時中止 $\hat{Q}_b^{prev}$ 的工具呼叫
    2. 負面採樣 (防止誤判):
        - $(\hat{Q}_b^T, \hat{Q}_b^{prev}) = \begin{cases} (\hat{Q}_b^T, \hat{Q}_b^{prev}), & 0.9 \text{ (正確)} \\ (\hat{Q}_b^T, \hat{Q}_b^{neg}), & 0.1 \text{ (替換為錯誤)} \end{cases}$

Think While Watching (arXiv 2603.11896)

MLLM 在處理影片時會因長度增加而產生記憶衰退，本研究採用 Think While Watching，在互動期間維持記憶。

Think While Watching — Streams and Turns

- Streams and Turns
  - Segmented stream: 將影片表示為一個有序的區段序列 $S_{1:T} \triangleq \{S_1, \ldots, S_T\}$
  - Multi-turn questioning: 使用者在第 r 次問問題 q_r 時，τ_r 代表問題索引，滿足 $1 \leq \tau_1 \leq \tau_2 \leq \cdots \leq \tau_R \leq T$，且每次問題系統只看到 $S_{1:\tau_r} \triangleq (S_1, \ldots, S_{\tau_r})$，對話歷史為 $\mathcal{H}_{r-1} \triangleq \langle (q_1, a_1), \ldots, (q_{r-1}, a_{r-1}) \rangle$

Think While Watching — Streaming Protocols / Units

- Streaming Protocols:
  - Single-round streaming, $R = 1$，產出思考過程 π_1 與輸出 a_1
  - Multi-round streaming, $R > 1$，每次根據 $S_{1:\pi_r}$ 與 $\mathcal{H}_{r-1}$ 產出 (π_r, a_r)
- Streaming Units:
  - 接收單元 $R_{1:U} \triangleq (R_1, \ldots, R_U)$，其中 $R_u \in \{S, Q\}$
  - 生成單元 $C_{1:U} \triangleq (C_1, \ldots, C_U)$ 一對一對齊:
    - 若 $R_u = S_t \to C_u = m_t$ (Memory Note)
    - 若 $R_u = Q_r \to C_u = (\pi_r, a_r)$
  - 位置跨度: $\Delta[R_u] = \begin{cases} \max\{T_u, H_u, W_u\}, & R_u \in \{S\} \\ L[R_u], & R_u \in \{Q\} \end{cases}$

Think While Watching — 方法：Segment-level Memory Notes

- 方法:
  - Segment-level Memory Notes:
    - $S_{1:t}$ 的記憶庫為 $\mathcal{M}_t \triangleq \{(i, m_i)\}_{i=1}^{t}$
    - 每個筆記 $m_t = \mathrm{Mem}_\theta[\mathbf{S}_t], \quad C_{\mathrm{idx}[S_t]} = m_t$，$\theta$ 為神經網路權重
    - 問題 q_r 時，透過 attention 生成 $\langle \pi_r, a_r \rangle \sim p_\theta[\pi_r, a_r \mid q_r, \mathcal{H}_{r-1}, \mathcal{M}_{\tau_r}]$

Think While Watching — Streaming Architecture

- Streaming Architecture:
  - Prefix and Suffix Formatting(for SFT):
    - Source Prefix: $\langle R_1, \ldots, R_U \rangle$
    - Target Suffix: $\langle C_1, \ldots, C_U \rangle$
  - Segment-level Streaming Attention Mask(preventing data leakage):
    - A 為提供 Query token 的片段，B 為提供 Key/Value token 的片段 ($A, B \in \{R_1, \ldots, R_U, C_1, \ldots, C_U\}$)
    - $$M^{seg}[A, B] = \begin{cases} \mathbb{I}[v \leq u], & A = R_u, B = R_v \\ \mathbb{I}[v \leq u], & A = C_u, B = R_v \\ \mathbb{I}[k \leq u], & A = C_u, B = C_k \\ 0, & \text{otherwise.} \end{cases}$$

Think While Watching — Streaming Positional Encoding with MRoPE

- Streaming Positional Encoding with MRoPE:
  - R: Standardized Cumulative offset (標準累積偏移量)
  - $Z_t = \frac{X_t - \mu_0}{\sigma}$
  - C: encoding from 0
  - B 為 based offset:

$$B_k = \begin{cases} B_k^S = \sum_{u<idx[S_k]} \Delta[R_u] \\ B_k^Q = \sum_{u<idx[Q_k]} \Delta[R_u] \\ B_k^C = \begin{cases} 0, & k = 1 \\ \sum_{i=1}^{k-1} L[C_i], & k \geq 2 \end{cases} \end{cases}$$

Think While Watching — 3-stage training

- 3-stage training:
  1. 片段 memory note 並回答單輪詢問
  2. 多輪對話
  3. 針對長影片訓練
     - Long-term memory
     - Uncertainty handling
     - Distractor robustness

Think While Watching — Parallel Streaming Inference

- Parallel Streaming Inference:
  - Dual KV Cache 解耦持續影片輸入與 Autoregressive Decoding。
  - Adaptive Attention Backend:
    - Prefilling 階段 ($q_{len} = k_{len}$): 採用 Flash Attention。
    - Autoregressive 解碼階段 ($q_{len} = 1$): 切換至 Memory-Efficient Attention 並施加顯式串流面罩，確保低延遲與因果律。

Video Streaming Thinking (arXiv 2603.12262)

傳統 MLLM 在處理動態串流影片時，通常採用「先接收完整片段、再進行批量推理」的離線模式，無法達到即時（Real-time）的思考與決策。本研究提出 Video Streaming Thinking VST，實現在影片持續輸入的同時，即時進行增量式推理與動態思維鏈（CoT）生成。

Video Streaming Thinking — 問題 / 做法

問題:

- 串流影片的高吞吐量導致推理計算資源暴增與高延遲。
- 動態畫面上資訊隨時間演進，模型難以同時保持歷史脈絡並進行即時思考。

做法:

- Dynamic Frame-Thought Interleaving:
  1. 接收畫面區塊 $V_{1:t} \triangleq \{v_1, \ldots, v_t\}$，其中 v_t 為當前時間步的影像幀。
  2. 引入思考觸發器，使模型在解碼過程中動態決定是否插入思維區塊 π_t：$P(\text{Action}_t \mid V_{1:t}, \pi_{<t}) = \text{Softmax}(W_a \cdot h_t)$ 其中 $\text{Action}_t \in \{\text{THINK}, \text{EMIT_RESPONSE}, \text{SKIP}\}$。

Video Streaming Thinking — Incremental KV Cache Update

- Incremental KV Cache Update:
  1. 將影像特徵與思考軌跡分別儲存至獨立的注意力空間。
  2. 當新幀 v_{t+1} 入隊時，僅對新 token 進行 Prefill 計算，保持歷史 K, V 矩陣不變，降低 $O(T^2)$ 記憶體複雜度至 $O(T)$。
  3. Stream-Consistent Attention Mask: 確保當前生成的思維 token $\pi_{t,i}$ 只能關注到過去的影像 $V_{1:t}$ 與 $\pi_{<t}$，避免 Data Leakage。

Adversarial Challenges in Network Intrusion Detection Systems: Research Insights and Future Prospects (arXiv 2409.18736)

基於機器學習與深度學習的網路入侵檢測系統（NIDS）極易受到對抗性攻擊（Adversarial Attacks）的威脅。攻擊者透過在封包特徵中注入微小的擾動，即可誘使模型發生誤判（如將惡意流量標記為正常）。本研究系統性評估了黑箱與白箱攻擊對 NIDS 的破壞力，並提出未來防禦的方向。

Adversarial Challenges in NIDS — 問題 / 做法

問題:

- 對抗性擾動會破壞 NIDS 的決策邊界，導致高漏報率
- 現有防禦機制多針對特定攻擊設計，缺乏對未知對抗範式的泛化能力。

做法:

- Adversarial Attack Formulation (對抗性攻擊數學表達):
  1. 給定原始封包特徵 $\mathrm{x} \in \mathcal{X}$ 與真實標籤 $\mathrm{y} \in \{0, 1\}$（0: 正常, 1: 惡意）
  2. 攻擊者的目標為尋求擾動向量 δ，使得模型 f_θ 發生誤判，同時滿足特徵可行性約束（如封包大小不能為負）：

$$\arg\min_{\delta} \mathcal{L}(\mathrm{f}_\theta(\mathrm{x} + \delta), \mathrm{y}) \quad \text{s.t.} \quad \|\delta\|_\mathrm{p} \leq \epsilon \quad \text{and} \quad (\mathrm{x} + \delta) \in \mathcal{S}_\text{valid}$$

  其中 $\mathcal{S}_\text{valid}$ 為符合真實網路協定規範的合法特徵空間。

Adversarial Challenges in NIDS — Systemic Threat Modeling

- Systemic Threat Modeling (系統化威脅建模):
  1. 白箱攻擊（FGSM, PGD）：假設攻擊者完全掌握 NIDS 模型參數 θ，利用梯度方向計算最佳擾動 $\delta = \epsilon \cdot \text{sign}(\nabla_{\text{x}} \mathcal{L}(\theta, \text{x}, \text{y}))$。
  2. 黑箱攻擊（Query-based, Transferability）：利用代理模型（Substitute Model）生成對抗樣本，驗證其在未知目標 NIDS 上的遷移性。

Adversarial Challenges in NIDS — 部署前實踐建議

在部署基於機器學習的 NIDS 前應考慮的實踐建議：

- 對抗訓練 (Adversarial Training)：在訓練中引入合理的對抗干擾（非過度簡化的對抗生成），避免過度擬合非合理的數據。
- 集成學習與融合 (Ensemble Learning & Fusion)：結合不同架構與訓練方式的模型，利用獨立錯誤與多樣化決策邊界（如投票、平均、元學習）提升整體韌性。
- 堅固的特徵工程 (Robust Feature Engineering)：整合高階語義特徵而非低階/原始數據，提升模型的抽象能力與穩定度。
- 輸入預處理 (Input Preprocessing)：透過標準化、特徵縮放、雜訊過濾與輸入轉換，降低模型對微小惡意擾動的敏感度。
- 異常偵測 (Anomaly Detection)：建立多步驟防禦機制，利用異常過濾器優先標示出非分佈（OOD）的可疑輸入。
- 正確且廣泛的評估 (Proper Evaluation)：使用包含邊緣案例（Corner cases）與極端狀況的即時資料集進行壓力測試，而非僅依賴過時的標準數據集

A Defensive Framework Against Adversarial Attacks on Machine Learning-Based Network Intrusion Detection Systems (arXiv 2502.15561)

針對 NIDS 面臨對抗性流量欺騙的問題，本研究提出了一套多層次的防禦框架，結合對抗訓練（Adversarial Training）、資料平衡（Data Balancing）與特徵選擇，著重提升模型在遭受到抗樣本擾動時的召回率（Recall）與整體穩定度。

Defensive Framework — 問題

問題:

1. 傳統 NIDS 模型在未受保護時遭遇對抗攻擊，效能會發生災難性潰敗（如對抗召回率暴跌至 10% 以下）。
2. 網路流量資料集常伴隨嚴重的類別不平衡問題，進一步削弱了模型對少數惡意攻擊的識別能力。

Defensive Framework — 做法：Multi-Stage Defensive Pipeline

做法:

- Multi-Stage Defensive Pipeline (多階段防禦管道):
  1. Adversarial Training & Balancing (對抗訓練與資料平衡):
     - 使用 SMOTE 演算法進行少數類別過採樣。
     - 在訓練集中混入由 PGD/FGSM 生成的對抗樣本 $x_{adv} = x + \delta$，優化極小極大化目標函數（Minimax Objective）：

$$\min_{\theta} \mathbb{E}_{(x,y)\sim\mathcal{D}} \left[\max_{\|\delta\| \leq \epsilon} \mathcal{L}(f_\theta(x+\delta), y) \right]$$

  2. Feature Engineering & Preprocessing (特徵工程與預處理):
     - 過濾高敏感、易被攻擊者操縱的次要特徵，僅保留判別力強且具備協定約束的關鍵欄位（如流速、封包持續時間）。
  3. Model Fine-Tuning (模型微調與組合):
     - 分別建立傳統機器學習組合（TC, 如 RF + XGBoost）與深度學習組合（DL, 如 MLP + CNN）。
     - 超參數調優以確保在「正常流量準確率」與「對抗流量召回率」之間取得最佳平衡。